

Docker Swarm: Introduction

📌 Introduction à Docker Swarm:

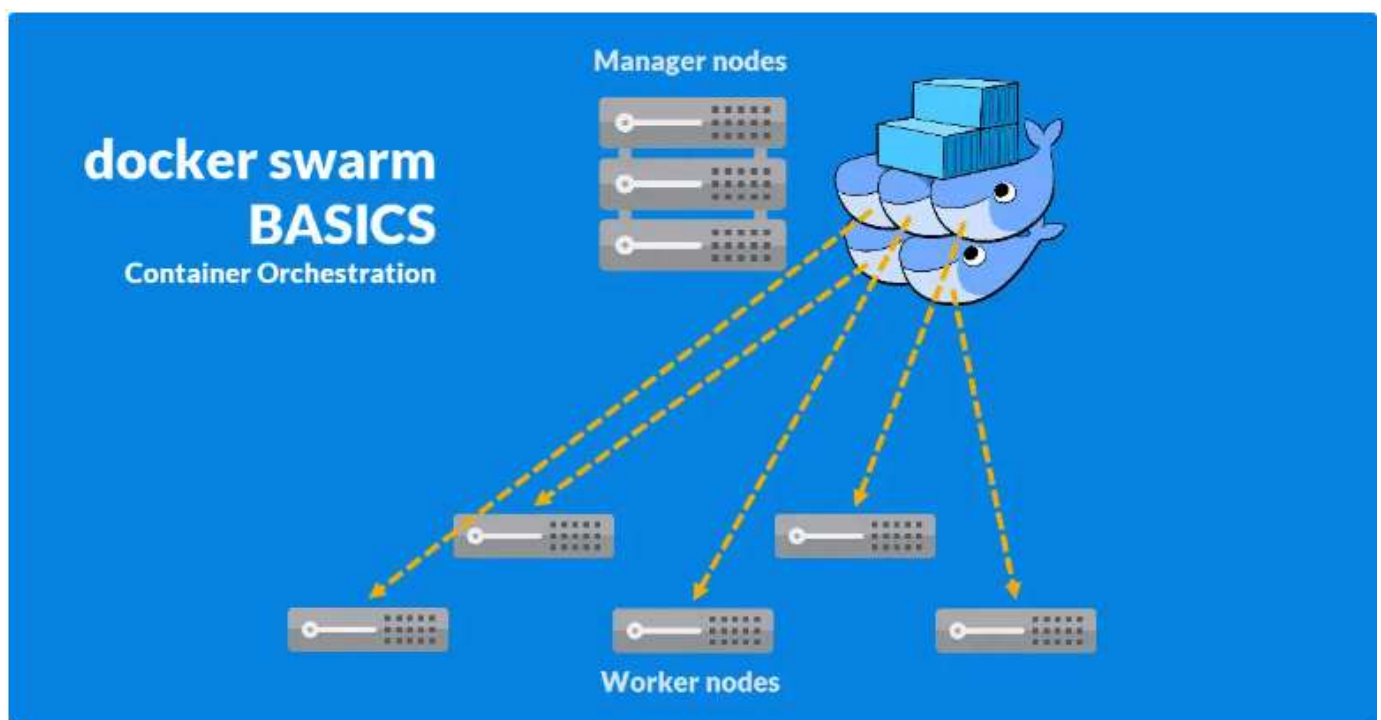
Dans le chapitre précédent, nous avons exploré **Docker Compose**, un outil permettant de définir et gérer des applications multi-conteneurs avec un fichier YAML. Docker Compose facilite le déploiement et la gestion des services sur une seule machine, en orchestrant plusieurs conteneurs de manière déclarative.

Cependant, dans un environnement de production, nous avons souvent besoin de **scalabilité**, de **haute disponibilité** et de **gestion de plusieurs nœuds**. C'est ici qu'intervient **Docker Swarm**, l'outil d'orchestration natif de Docker.

Avec **Docker Swarm**, nous passons d'une gestion locale des conteneurs à une gestion distribuée sur plusieurs machines (nœuds). Il permet :

- ◆ **Le clustering** de plusieurs hôtes Docker pour former un "Swarm".
- ◆ **La répartition de charge** et la haute disponibilité.
- ◆ **Le scaling automatique** des services.
- ◆ **La tolérance aux pannes**, avec des stratégies de redémarrage et de remplacement.

Alors que **Docker Compose** est idéal pour le développement et les tests locaux, **Docker Swarm** est conçu pour le déploiement en production et la gestion d'infrastructures plus complexes.



Docker Swarm reprend plusieurs principes de **Docker Compose**, ce qui facilite la transition. Nous allons voir comment passer de docker-compose up à docker stack deploy pour gérer nos services sur un cluster. 🚀

Qu'est-ce que Docker Swarm ?

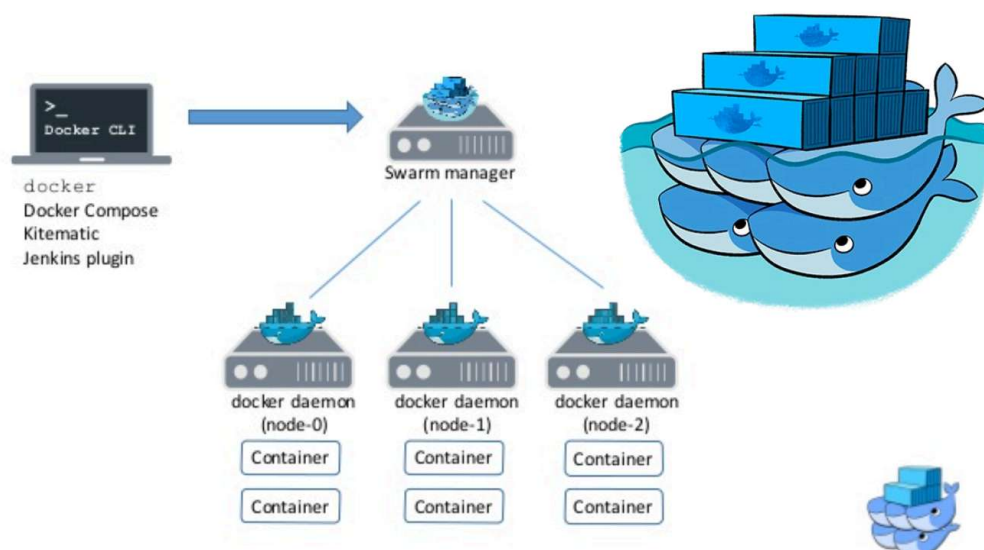
Définition et rôle:

Définition:

Docker Swarm est la solution native et officielle *d'orchestration de conteneurs de Docker*. Il permet de transformer un ensemble d'hôtes Docker en un seul hôte virtuel, donnant ainsi l'impression qu'ils fonctionnent sur une seule machine. Swarm repose sur une API unifiée, ce qui signifie que tous les outils interagissant avec le démon Docker (comme Docker Compose, Docker Desktop, Jenkins, et bien sûr le client Docker) peuvent être utilisés de manière transparente avec Docker Swarm.

Rôles d'un Docker Swarm :

- ◆ **Clustering** : Docker Swarm permet de regrouper plusieurs hôtes Docker pour les faire fonctionner ensemble en tant qu'un seul cluster. Cela simplifie la gestion et la distribution des conteneurs dans un environnement distribué.
- ◆ **Ordonnancement de conteneurs** : Il assure l'ordonnancement des tâches (containers) sur les différents nœuds du cluster en fonction de la capacité disponible. Par défaut, il utilise un mécanisme appelé **Bin Packing** (empilement des conteneurs de manière optimale pour maximiser l'utilisation des ressources des nœuds).
- ◆ **Haute Disponibilité et Répartition de Charge** : Docker Swarm offre une haute disponibilité grâce à la tolérance aux pannes et à la répartition de la charge entre les différents nœuds du cluster, garantissant ainsi une performance optimale même en cas de défaillance d'un nœud.



Docker Swarm vs Kubernetes:

Maintenant que nous avons défini **Docker Swarm** et ses fonctionnalités, il est utile de le comparer à une autre solution d'orchestration largement utilisée dans l'industrie : **Kubernetes**, afin de mieux comprendre leurs points forts et leurs différences.

Caractéristique	Docker Swarm	Kubernetes
Intégration dans Docker	Intégré nativement dans Docker	N'est pas intégré dans Docker, nécessite un environnement séparé
Installation & Configuration	Facile à installer, homogène sur tous les OS, plus facile à apprendre	Installation complexe, varie selon l'OS, courbe d'apprentissage plus raide
Déploiement d'application	Utilise YAML ou Docker Compose pour les services	Supporte namespaces, pods et déploiements
Disponibilité & Scalabilité	Haute disponibilité, pas de scaling automatique, déploiement rapide	Haute disponibilité, scaling automatique, auto-réparation
Surveillance	Pas de surveillance intégrée, outils tiers nécessaires	Surveillance intégrée, support des outils tiers
Sécurité	Utilise TLS pour le contrôle d'accès	Supporte RBAC, SSL/TLS, gestion des secrets
Équilibrage de charge	Équilibrage de charge automatique via DNS	Pas d'équilibrage automatique, nécessite Nginx Ingress

➡ En résumé, **Docker Swarm** est idéal pour les débutants et les entreprises avec des besoins simples, offrant une gestion facile des conteneurs à grande échelle. Si vous avez des projets complexes nécessitant des fonctionnalités avancées comme la surveillance, la sécurité et la haute disponibilité, **Kubernetes** est la solution à privilégier.

Architecture: Nœuds worker et manager

📌 Introduction:

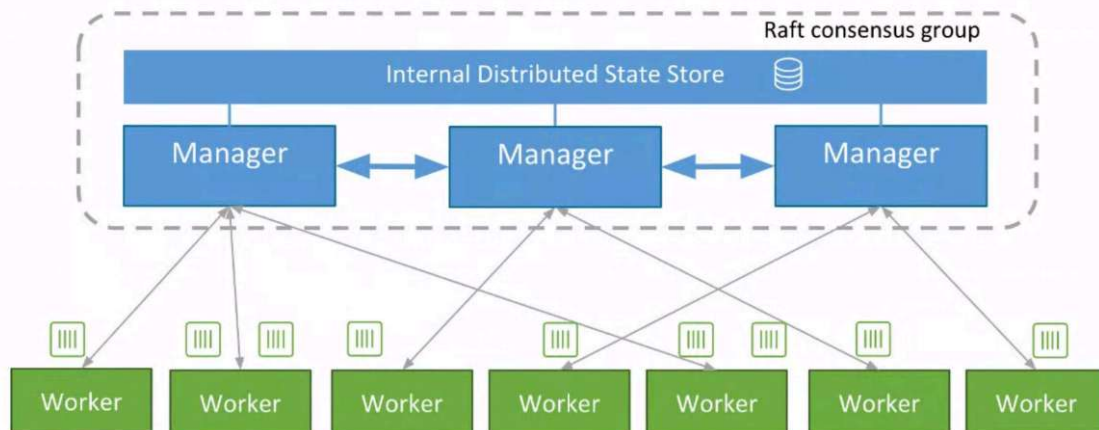
Dans cette partie, nous explorerons l'architecture de **Docker Swarm**, qui se compose de **nœuds managers (Manager Node)** et **nœuds workers (Worker Node)**. Ces composants collaborent pour gérer et orchestrer les conteneurs dans un cluster. Comprendre cette structure est essentiel pour utiliser Docker Swarm efficacement dans des déploiements à grande échelle.

📌 Les types de nœuds dans un Docker Swarm:

L'architecture de **Docker Swarm** repose sur un ensemble de nœuds (des machines physiques ou virtuelles). Ces nœuds se composent de nœuds **Manager** et de nœuds **Worker**.

Dans un **Docker Swarm**, il y a généralement plusieurs nœuds workers et au moins un nœud manager, qui est responsable de la gestion des ressources des nœuds workers et de garantir que le cluster fonctionne de manière optimale.

Swarm Architecture



1- Nœuds Manager (Manager Node):

1.1 Définition et rôles:

Les **nœuds Manager** sont essentiels pour la gestion et l'orchestration du cluster Docker Swarm. Voici leurs principales responsabilités :

- **Gestion de l'État du Cluster** : Maintiennent l'état global du cluster, assurant la cohérence et la synchronisation entre tous les nœuds.

- **Planification des Services** : Déterminent le placement des services sur les nœuds Workers et gèrent la mise à l'échelle des applications.
- **API HTTP en Mode Swarm** : Exposent une API pour interagir avec le cluster, permettant le déploiement et la gestion des services.
- **Communication avec les Nœuds Workers** : Distribuent les tâches d'exécution aux nœuds Workers sans qu'ils prennent de décisions sur l'état global du cluster.
- **Haute Disponibilité** : Garantissent la tolérance aux pannes en utilisant plusieurs nœuds Managers, idéalement en nombre impair pour maintenir le consensus.

1.2 Composants du Nœud Manager:

1 API Docker :

- Accepte les commandes des utilisateurs ou des outils externes.
- Crée et gère les objets de service (services et configurations).

2 Orchestrator :

- Boucle de réconciliation qui compare l'état actuel du cluster avec l'état souhaité.
- Crée des tâches pour s'assurer que le nombre de réplicas et l'état du cluster correspondent à la définition du service.

3 Allocator :

- Assigne des adresses IP aux tâches et conteneurs pour la communication réseau.

4 Dispatcher :

- Assigne les tâches aux nœuds Worker et s'assure qu'ils reçoivent les mises à jour des services.

5 Scheduler (Planificateur) :

- Décide quel nœud Worker exécute une tâche en fonction des ressources disponibles et des règles de placement.

6 Raft (Composant de Consensus) :

- Assure la cohérence du cluster en élisant un **Leader Node** parmi les nœuds Manager. Raft gère la répartition des tâches, la synchronisation de l'état du cluster et permet de prendre des décisions critiques de manière cohérente. Si le Leader échoue, Raft élit automatiquement un nouveau Leader pour maintenir la stabilité du Swarm.

2- Nœuds Worker (Worker Node):

2.1 Définition et rôles:

Les **nœuds Worker** sont responsables de l'exécution des conteneurs dans le cluster. Voici leurs principales fonctions :

- **Exécution des Conteneurs** : Ils lancent et gèrent les conteneurs assignés par les nœuds Manager.
- **Absence de Décisions** : Ils ne prennent pas part à la gestion du cluster ni à la planification des services.
- **Réception des Tâches** : Reçoivent les tâches à exécuter des nœuds Manager.

- **Flexibilité des Ressources** : Peuvent être ajoutés ou retirés en fonction des besoins du cluster.

2.2 Composants du Nœud Worker :

1 Worker Agent:

- Se connecte au **dispatcher** du nœud Manager pour récupérer les tâches qui lui sont assignées.

2 Executor:

- Exécute les **tâches** assignées en lançant des conteneurs Docker.
- Gère l'exécution et l'état des conteneurs en fonction des instructions du Manager.

 **Remarque:** Un Swarm peut avoir un seul nœud Manager, mais un nœud Worker nécessite toujours un nœud Manager.

Exemple théorique:

Utilisons cet exemple pour mieux comprendre comment ces différents composants communiquent entre eux et comment l'algorithme **Raft** intervient dans le processus :

1 Création du Service :

L'administrateur exécute :

```
docker service create --name web-app --replicas 3 -p 80:80 my-web-image
```

- Le Manager reçoit la commande et crée un service avec 3 réplicas. Si plusieurs nœuds Manager existent, un nœud est élu comme **Leader** via l'algorithme **Raft**, qui garantit la cohérence du cluster.

2 Planification et Attribution des Tâches :

- L'Orchestrator génère 3 tâches.
- Le Scheduler sélectionne 3 nœuds Worker disponibles.

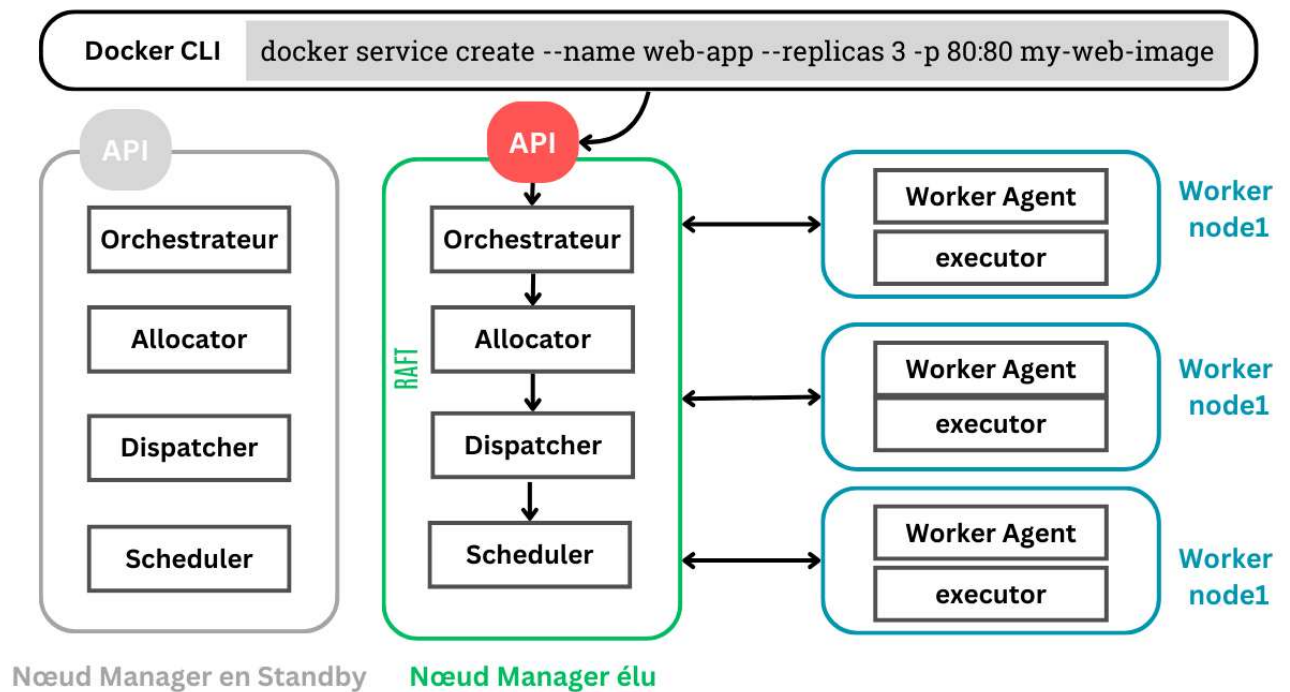
3 Exécution des Conteneurs :

- Le Dispatcher assigne les tâches aux Workers.
- Les Workers exécutent les conteneurs et les rendent accessibles.

4 Surveillance et Auto-Réparation :

- Si un Worker tombe, le Manager, en s'appuyant sur Raft, recrée automatiquement la tâche sur un autre Worker. Raft garantit que tous les Managers partagent le même état et que le processus reste cohérent même en cas de défaillance.

✓ **Résultat** : Le service tourne toujours avec 3 instances disponibles, grâce à la gestion cohérente assurée par l'algorithme Raft et l'auto-réplication des tâches.



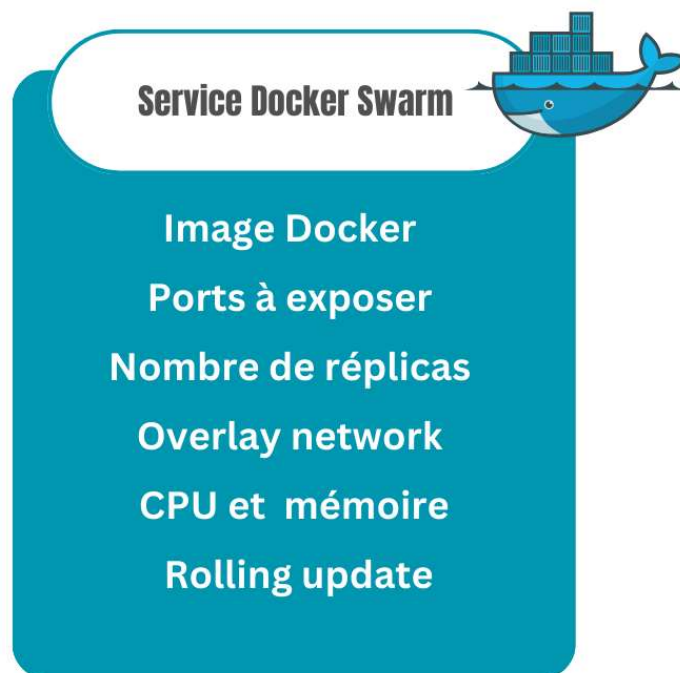
Architecture: Services et tâches

Services et tâches:

1- Les Services:

Un **service** sous Docker Swarm définit la manière dont un ou plusieurs conteneurs vont fonctionner au sein du cluster. En effet pour créer un service il faut spécifier:

- **L'image Docker** à utiliser pour les conteneurs.
- **Les ports** à exposer pour la communication avec l'extérieur.
- **Le nombre de réplicas** (instances) du service à exécuter, pour garantir la disponibilité et la scalabilité.
- **Un réseau overlay** pour que le service se connecte à d'autres services dans le swarm
- **Limites et réservations de CPU et de mémoire**
- **Une politique de mise à jour progressive** (rolling update)

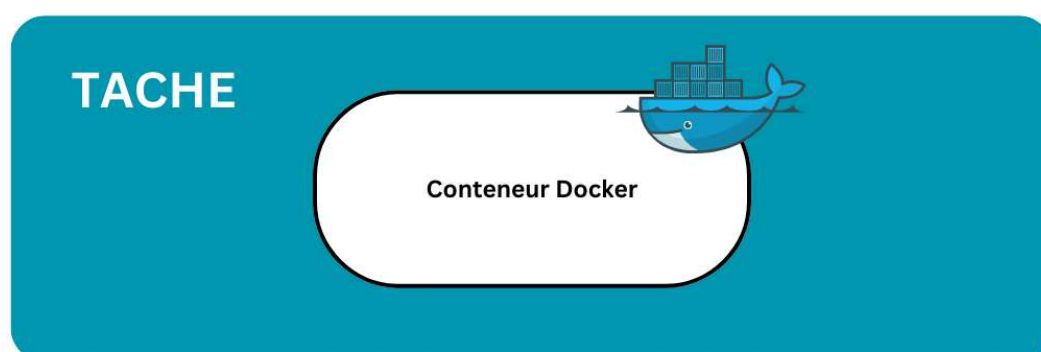


2- Les Tâches (Tasks):

Une **tâche** dans Docker Swarm est une **instance de conteneur** lancée par un **service** sur un **nœud Worker** dans un cluster Docker Swarm. Chaque tâche représente un conteneur individuel qui exécute un processus spécifique, tel qu'une application ou un service, en fonction de la configuration du service.

Les tâches progressent dans un flux unidirectionnel à travers plusieurs états :

- 1 **Assignée** (Assigned) : La tâche a été attribuée à un nœud du Swarm.
- 2 **Préparée** (Preparing) : La tâche est en cours de préparation pour l'exécution, y compris la configuration du conteneur.
- 3 **En cours d'exécution** (Running) : Le conteneur est en cours d'exécution et la tâche accomplit sa fonction.
- 4 **Terminée** (Complete) : La tâche a terminé son exécution avec succès.
- 5 **Échouée** (Failed) : La tâche a échoué, soit en raison d'un échec du conteneur, soit à cause d'un échec des contrôles de santé.
- 6 **Remplacée** (Replaced) : Si une tâche échoue, une nouvelle tâche est créée pour la remplacer, afin de maintenir l'état souhaité du service.

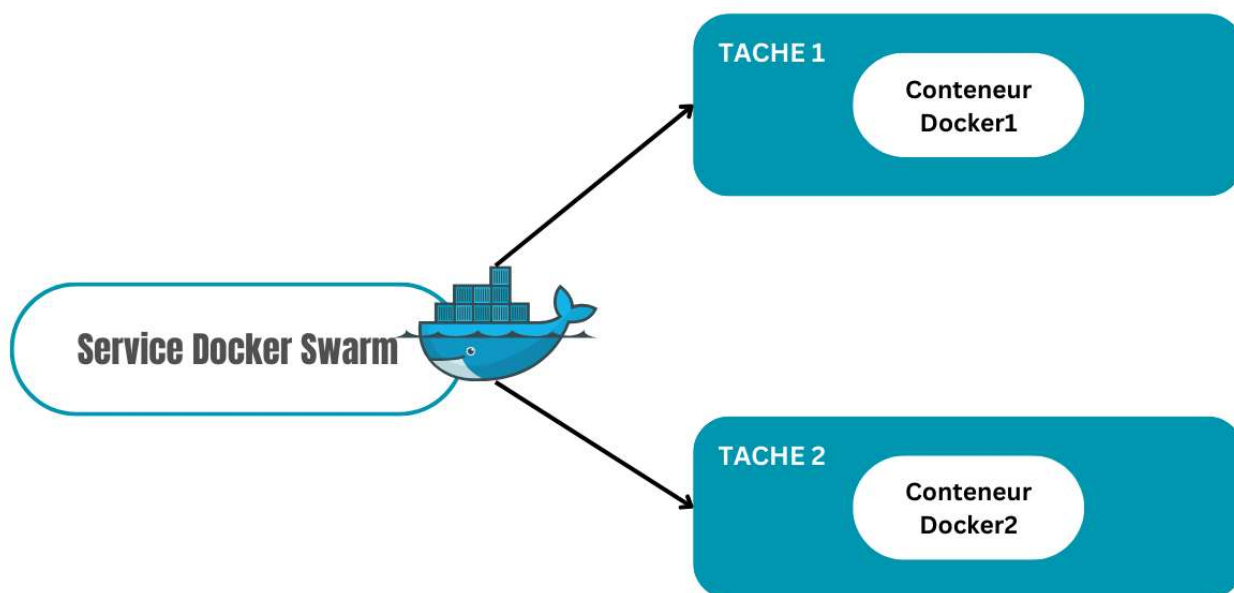


3- La relation entre Service, Tâches et Conteneurs:

Lorsqu'un service est déployé sur **Docker Swarm**:

- Le gestionnaire de swarm reçoit la définition du service, incluant le nombre de répliques souhaitées, les contraintes, les ressources nécessaires, etc.
- Cela constitue l'état désiré pour ce service.

- Le gestionnaire planifie ce service sur les nœuds du swarm sous forme de **tâches** (tasks), qui sont des unités de travail indépendantes représentant l'exécution d'un conteneur.
- Chaque **tâche** correspond à l'exécution d'un conteneur sur un nœud spécifique. Si le service a plusieurs répliques, plusieurs tâches (et donc plusieurs conteneurs) seront créées pour maintenir le nombre de répliques spécifié. Chaque tâche est indépendante, mais elles forment toutes le **service** dans son ensemble.



Introduction

C'est quoi le Swarm mode?

Le **Swarm mode** est une fonctionnalité intégrée de Docker qui permet de créer et de gérer un cluster de machines Docker. Lorsqu'un groupe d'hôtes Docker est configuré pour fonctionner en **mode swarm**, ces hôtes forment un cluster et coopèrent pour exécuter des applications conteneurisées. Ce mode permet de déployer, de scaler et de gérer des services de manière hautement disponible et évolutive, tout en offrant des fonctionnalités telles que la répartition automatique des charges et la tolérance aux pannes. Swarm mode permet ainsi à Docker Swarm de fonctionner de manière optimale pour l'orchestration de conteneurs sur plusieurs machines physiques ou virtuelles.

Ainsi, **Docker Swarm** fait référence à l'outil d'orchestration dans son ensemble, tandis que **Swarm mode** est le mode spécifique qui active le clustering et l'orchestration des services au sein du système Docker.

Exigences système pour le mode Swarm:

1 Machines physiques ou virtuelles:

Pour configurer un **Docker Swarm**, vous pouvez utiliser :

- **Des machines physiques** (serveurs, PC)
- **Des machines virtuelles** (via VirtualBox, VMware, ou le cloud)
- Si vous manquez de ressources, vous pouvez utiliser **Play with Docker** :  labs.play-with-docker.com (<https://labs.play-with-docker.com/>) est une plateforme en ligne qui permet de tester Docker Swarm sans installation locale. Elle offre un environnement prêt à l'emploi pour créer et gérer des conteneurs Docker directement depuis votre navigateur.

 **Rappel:** Un cluster Docker Swarm nécessite au minimum 1 nœud Manager et 1 nœud Worker.

2 Installation de Docker sur chaque nœud:

Pour que les nœuds puissent rejoindre le Swarm, **Docker doit être installé** sur chaque machine.

Les prochaines étapes dans cette section:

1. Configuration d'un Docker Swarm.
2. Déploiement d'une application.
3. Mise à l'échelle d'une application (Scaling).
4. Gestion des nœuds dans Swarm.
5. Nettoyage du Swarm.

💡 Une fois l'installation terminée, nous pouvons initialiser le Swarm et commencer à orchestrer vos

Gestion d'un Docker Swarm

Introduction:

Docker Swarm permet de regrouper plusieurs machines en un **cluster** fonctionnant comme un seul hôte Docker. Cette section vous guidera à travers les étapes essentielles pour:

- ✓ initialiser un Swarm
- ✓ Ajouter des nœuds (worker et manager) au Swarm.
- ✓ Vérifier que le Swarm est bien configuré et prêt à déployer des applications.
- ✓ Quitter un Swarm.

Initialiser un Swarm:

Pour initialiser un **Docker Swarm**, on utilise la commande suivante :

```
docker swarm init --advertise-addr <IP_MANAGER>
```

Explication:

1. **docker swarm init** : Cette commande transforme le nœud actuel en **gestionnaire (manager)** du cluster Swarm.
2. **--advertise-addr <IP_MANAGER>** : Ce paramètre spécifie l'adresse IP du nœud qui servira de **gestionnaire principal**. Cela est nécessaire si le serveur a plusieurs interfaces réseau.
3. Sur le serveur où vous exécutez la commande, il devient le **nœud manager** principal.

Exemple:

1- Lancer cette commande pour récupérer l'adresse IP du nœud qui servira de gestionnaire principal:

```
ifconfig
```

```

rihem@rihem-virtual-machine: $ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 02:42:c9:c5:e0:9c txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker_gwbridge: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 02:42:a1:05:81:d4 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 85 bytes 14614 (14.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.127.132 netmask 255.255.255.0 broadcast 192.168.127.255
    inet6 fe80::165e:1386:2255:f825 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:1b:a4:cd txqueuelen 1000 (Ethernet)
    RX packets 1126 bytes 370402 (370.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 931 bytes 103575 (103.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

2-Lancer un Docker Swarm en utilisant cette commande:

```
docker swarm init --advertise-addr 192.168.127.132
```

```

rihem@rihem-virtual-machine: $ docker swarm init --advertise-addr 192.168.127.132
Swarm initialized: current node (no2af8bhyo0sa8m42ppmmgf58) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3igk2hf5t2zi7lh9om022znjqpc51xt68sc4tebydhboscldo-8l2ocny8di4ezkrf13m53p1tm 192.168.127.132:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
rihem@rihem-virtual-machine: $

```

💡 Remarque :

- Cette commande doit être exécutée sur un **nœud maître**.
- Les autres nœuds doivent rejoindre le cluster avec `docker swarm join`.

📌 Ajouter des nœuds au Swarm:

1- Ajout de nœuds Worker à un Swarm:

Pour ajouter un nœud Worker à un Swarm, on utilise la commande suivante:

```
docker swarm join --token SWMTKN-1-xxxxx 192.168.1.100:2377
```

📖 Explication :

1. Ce serveur rejoint le cluster en tant que **worker**.
2. Le **token** assure l'authentification et l'autorisation d'accès.
3. 192.168.1.100:2377 est l'adresse IP et le port du **manager**.

Si vous voulez ajouter un **autre worker**, exécutez d'abord cette commande sur le **manager principal** pour récupérer le token des workers:

```
docker swarm join-token worker
```

Exemple:

1- Ajouter un noeud worker au swarm:

```
rihem@rihem-virtual-machine:~$ docker swarm join --token SWMTKN-1-3fp1n1fsr810xahwpf5n46l0kmj73b6p7vu7vjvzmjwh1xr0ri-90x5zda2axbtiq2jtiqadpq78 192.168.127.132:2377
This node joined a swarm as a worker.
```

2- Ajouter un autre noeud worker au swarm:

- Générer le token du noeud worker à partir du noeud principal:

```
rihem@rihem-virtual-machine:~$ docker swarm join-token worker
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3fp1n1fsr810xahwpf5n46l0kmj73b6p7vu7vjvzmjwh1xr0ri-90x5zda2axbtiq2jtiqadpq78 192.168.127.132:2377
```

- Lancer cette commande dans le terminal du noeud worker en question:

```
rihem@rihem-virtual-machine:~$ docker swarm join --token SWMTKN-1-3fp1n1fsr810xahwpf5n46l0kmj73b6p7vu7vjvzmjwh1xr0ri-90x5zda2axbtiq2jtiqadpq78 192.168.127.132:2377
This node joined a swarm as a worker.
```

2- Ajout de nœuds Manager à un Swarm:

- Si vous voulez ajouter un **autre manager** (ex. pour une haute disponibilité), exécutez d'abord cette commande sur le **manager principal** pour récupérer le token des managers :

```
docker swarm join-token manager
```

```
rihem@rihem-virtual-machine:~$ docker swarm join-token manager
To add a manager to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3fp1n1fsr810xahwpf5n46l0kmj73b6p7vu7vjvzmjwh1xr0ri-2ed8hp4spo5r16e8obvgd6h0k 192.168.127.132:2377
```

- Sur un autre serveur qui doit devenir **manager**, exécutez cette commande :

```
docker swarm join --token SWMTKN-2-yyyyy 192.168.1.100:2377
```



```
rihem@rihem-virtual-machine:~$ docker swarm join --token SWMTKN-1-3fp1n1fsr810xahwpf5n46l0kmj73b6p7vu7vjvzmjwh1xr0ri-2ed8hp4spo5r16e8obvgd6h0k 192.168.127.132:2377
This node joined a swarm as a manager.
```

Explication :

- Ce serveur rejoint le cluster en tant que **manager**.
- Il pourra **gérer** et **planifier** des services, en plus des tâches d'un worker.

Vérifier l'état d'un Swarm:

Pour vérifier l'état d'un cluster du Swarm il suffit d'exécuter cette commande:

```
docker info
```

```
Swarm: active
NodeID: qmljcg6ffqh9mxo7mbtipw4do
Is Manager: true
ClusterID: 0v2txt3jhztm9op1b28cll1e7
Managers: 2
Nodes: 4
Data Path Port: 4789
Orchestration:
  Task History Retention Limit: 5
Raft:
  Snapshot Interval: 10000
  Number of Old Snapshots to Retain: 0
  Heartbeat Tick: 1
  Election Tick: 10
Dispatcher:
  Heartbeat Period: 5 seconds
CA Configuration:
  Expiry Duration: 3 months
  Force Rotate: 0
Autolock Managers: false
Root Rotation In Progress: false
Node Address: 192.168.127.132
Manager Addresses:
  192.168.127.131:2377
  192.168.127.132:2377
```

Quitter un Swarm:

Pour quitter le Swarm on exécute la commande suivante:

```
docker swarm leave
```

Explication :

- Cette commande est exécutée depuis un Worker ou un Manager non leader pour faire quitter le nœud du Swarm.

Pour désactiver complètement le Swarm (depuis le Manager leader) on exécute la commande suivante:

```
docker swarm leave --force
```

⚠ *Faites attention: Cette commande **désactivera le Swarm sur ce nœud** et pourrait rendre le cluster inutilisable.*

💻 **Exemple:**

1- Quitter le Swarm en tant que worker node:

```
rihem@rihem-virtual-machine:~$ docker swarm leave
Node left the swarm.
```

2- Quitter le swarm en tant que manager node:

```
rihem@rihem-virtual-machine:~$ docker swarm leave
Error response from daemon: You are attempting to leave the swarm on a node that is participating as a manager. Removing this node leaves 1 managers out of 2. Without a Raft quorum your swarm will be inaccessible. The only way to restore a swarm that has lost consensus is to reinitialize it with '--force-new-cluster'. Use '--force' to suppress this message.
```

Gestion des Nœuds

Introduction:

Dans un cluster **Docker Swarm**, les nœuds jouent un rôle essentiel dans l'orchestration et l'exécution des conteneurs. Un nœud peut être soit un **manager** (responsable de la gestion du cluster et de la répartition des tâches), soit un **worker** (chargé d'exécuter les services).

Cette section couvre les principales opérations de gestion des nœuds, notamment :

- ◆ **Ajout et suppression de nœuds** dans le cluster.
- ◆ **Promotion et rétrogradation des nœuds** (worker $\rightleftharpoons$ manager).
- ◆ **Affichage de l'état des nœuds** et diagnostic du cluster.
- ◆ **Suppression ou remplacement d'un nœud défaillant**.

Lister tous les nœuds du Swarm:

Pour lister les nœuds d'un Swarm, on utilise la commande suivante :

```
docker node ls
```

```
rihem@rihem-virtual-machine: $ docker node ls
ID                                HOSTNAME                STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
4i044dkeixc26flwsj7j6gdpt        rihem-virtual-machine   Ready    Active                               27.5.1
6489fjmyrt76ug11vqre8rzh *      rihem-virtual-machine   Ready    Active    Leader           27.5.1
p5mxw9qkb4s1volka8soc4yfn        rihem-virtual-machine   Ready    Active                               27.5.1
```

Explication :

- Cette commande permet d'afficher **tous les nœuds** (Managers et Workers) du cluster Swarm.
- Elle montre des informations clés comme :
 - **ID** du nœud
 - **Nom** du nœud
 - **Rôle** du nœud (Manager ou Worker)
 - **État** (Ready, Down, etc.)
 - **Disponibilité** (Active, Drain, Pause)
 - **Leader** (indique quel Manager est le leader du cluster)

Remarque: L'astérisque * indique **le nœud sur lequel la commande a été exécutée**.



Mise à jour des nœuds dans Docker Swarm:

- Afin de modifier les propriétés d'un nœud dans un cluster Swarm, comme sa disponibilité, ses labels ou son rôle on utilise la commande:

```
docker node update [OPTIONS] <nom_du_nœud/ID>
```

- Voyons les différentes options qu'on peut utiliser avec cette commande:

Option	Description
--availability active	Définit le nœud comme actif (peut exécuter des tâches).
--availability pause	Met le nœud en pause (il ne reçoit plus de nouvelles tâches, mais garde celles en cours).
--availability drain	Vide le nœud de toutes ses tâches et empêche l'attribution de nouvelles.
--label-add key=value	Ajoute un label personnalisé au nœud.
--label-rm key	Supprime un label existant.



Exemples à pratiquer:

1- Ajouter un label au nœud:

```
docker node update --label-add region=africa <ID/nom>
```

```
rihem@rihem-virtual-machine:~$ docker node update --label-add region=africa 4i044dkeixc26flwsj7j6gdpt
4i044dkeixc26flwsj7j6gdpt
```

2- Vérifier la modification:

```
docker node inspect <ID/Nom>
```

```
rihem@rihem-virtual-machine:~$ docker node inspect 4i044dkeixc26flwsj7j6gdpt
[
  {
    "ID": "4i044dkeixc26flwsj7j6gdpt",
    "Version": {
      "Index": 35
    },
    "CreatedAt": "2025-02-10T14:38:47.584099149Z",
    "UpdatedAt": "2025-02-10T15:25:36.260827751Z",
    "Spec": {
      "Labels": {
        "name": "worker1",
        "region": "africa"
      },
      "Role": "worker",
      "Availability": "active"
    }
  }
]
```

Explication:

- Les **labels** aident à **catégoriser les nœuds** en fonction de leur **rôle, emplacement, matériel et environnement**. Ils permettent de mieux gérer le déploiement des services et d'optimiser l'orchestration.
- Dans ce cas, nous avons catégorisé le nœud Worker en fonction de la région **Africa**.

Voici d'autres labels que vous pouvez utiliser:

- **role**: Définir le rôle du nœud
- **environment**: Séparer les environnements (par exemple dev, staging, prod etc...)
- **hardware**: Spécifier le matériel
- **os**: Définir le type d'OS
- **datacenter**: Identifier le centre de données
- **backup**: Spécifier si le nœud est pour la sauvegarde
- **security_level**: Classifier par niveau de sécurité.

3-Mettre un nœud en mode maintenance (drain):

- Inspecter initialement les noeuds:

```
docker node ls
```

```
rihem@rihem-virtual-machine: $ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
4i044dkeixc26flwsj7j6gdpt	rihem-virtual-machine	Ready	Active		27.5.1
6489fjmyrt76ug11vqre8rzh *	rihem-virtual-machine	Ready	Active	Leader	27.5.1
p5mxw9qkb4s1volka8soc4yfm	rihem-virtual-machine	Ready	Active		27.5.1

- Mettre à jour le noeud en question:

```
docker node update --availability drain <ID/nom>
```

```
rihem@rihem-virtual-machine: $ docker node update --availability drain 4i044dkeixc26flwsj7j6gdpt
4i044dkeixc26flwsj7j6gdpt
```

- Vérifier de nouveau :

```
docker node ls
```

```
rihem@rihem-virtual-machine: $ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
4i044dkeixc26flwsj7j6gdpt	rihem-virtual-machine	Ready	Drain		27.5.1
6489fjmyrt76ug11vqre8rzh *	rihem-virtual-machine	Ready	Active	Leader	27.5.1
p5mxw9qkb4s1volka8soc4yfm	rihem-virtual-machine	Ready	Active		27.5.1



Inspecter un nœud spécifique:

Pour Inspecter un nœud spécifique on utilise cette commande:

```
docker node inspect <nom_du_noeud/ID>
```

```
rthen@rthen-virtual-machine: $ docker node inspect 4l044dketxc26flwsj7j6gdpt
[
  {
    "ID": "4l044dketxc26flwsj7j6gdpt",
    "Version": {
      "Index": 20
    },
    "CreatedAt": "2025-02-10T14:38:47.584099149Z",
    "UpdatedAt": "2025-02-10T14:38:47.681663304Z",
    "Spec": {
      "Labels": {},
      "Role": "worker",
      "Availability": "active"
    },
    "Description": {
      "Hostname": "rthen-virtual-machine",
      "Platform": {
        "Architecture": "x86_64",
        "OS": "linux"
      },
      "Resources": {
        "NanoCPUs": 2000000000,
        "MemoryBytes": 4056539136
      },
      "Engine": {
        "EngineVersion": "27.5.1",
        "Plugins": [
          {
            "Type": "Log",
            "Name": "awslogs"
          },
          {
            "Type": "Log",
            "Name": "fluentd"
          },
          {
            "Type": "Log",

```

Explication :

- Cette commande permet d'afficher **toutes les informations** sur un nœud spécifique.
- Elle fournit des détails tels que :
 - L'adresse IP du nœud
 - Son rôle (Manager ou Worker)
 - Son statut dans le cluster
 - Les labels assignés



Promouvoir un Worker en Manager:

- Promouvoir un worker en manager dans ce cas, veut dire transformer un **Worker** en **Manager en utilisant la commande suivante:**

```
docker node promote <nom_du_worker/ID>
```

Déploiement et Scaling Des applications sur Docker Swarm: docker service

Introduction:

Dans cette section, nous allons explorer comment déployer et faire évoluer (scaler) des applications au sein d'un cluster Docker Swarm. Docker Swarm offre une solution simple et puissante pour gérer des applications distribuées, permettant de lancer des services sur plusieurs nœuds tout en assurant leur disponibilité et évolutivité.

Nous verrons comment :

- **Déployer des applications** en utilisant la commande ***docker service***.
- **Faire évoluer les applications** en ajustant dynamiquement le nombre de réplicas pour répondre aux besoins en ressources.

La commande `docker service` :

Dans Docker Swarm, les applications sont déployées sous forme de **services**. **Les services Swarm** utilisent un modèle déclaratif, ce qui signifie que vous définissez l'état souhaité du service, et Docker prend en charge le maintien de cet état. Cet état inclut des informations telles que le nombre de réplicas, les ressources allouées, les ports exposés, et d'autres configurations. Docker se charge automatiquement de garantir que l'état défini soit respecté, même en cas de défaillance ou de mise à jour du service.

Voici une liste des principales commandes ***docker service*** utilisées pour gérer les services dans Docker Swarm :

Commande	Description
docker service create	Crée un nouveau service dans Docker Swarm.
docker service ls	Liste tous les services en cours d'exécution dans le Swarm.
docker service ps	Affiche les tâches (réplicas) d'un service spécifique, y compris leur état actuel.
docker service inspect	Affiche les détails complets du service spécifié.

docker service update	Met à jour un service en modifiant des configurations comme les réplicas, les ports, les variables d'environnement, etc.
docker service scale	Modifie le nombre de réplicas d'un service sans changer sa configuration. Cette commande est obsolète au profit de docker service update --replicas.
docker service rm	Supprime un service du Swarm.
docker service logs	Affiche les journaux d'un service en particulier, utile pour le débogage.
docker service rollback	Annule la mise à jour du service et le ramène à son état précédent.

Déploiement d'un service avec `docker service` :

Le déploiement d'une application sur Docker Swarm consiste à déployer un service sur un cluster Swarm. Il s'agit de définir l'image de l'application, le nombre de réplicas souhaités et de s'assurer que le service est correctement configuré pour tourner sur le Swarm. Le déploiement est simple grâce à la commande **docker service create**, qui permet de gérer plusieurs instances de votre application sur différents nœuds du cluster.

Exemple à pratiquer:

Imaginons que nous souhaitons déployer une application web Nginx avec Docker Swarm.

1- Création d'un service avec 2 réplicas de l'application Nginx :

- Vérifier que tous les noeuds sont en mode Active:

```
rihem@rihem-virtual-machine: $ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
4i044dkeixc26flwsj7j6gdpt	rihem-virtual-machine	Ready	Active		27.5.1
6489fjmyrt76ug11vqre8rzh *	rihem-virtual-machine	Ready	Active	Leader	27.5.1
p5mxw9qkb4s1volka8soc4yfm	rihem-virtual-machine	Ready	Active		27.5.1

- Créer le service:

```
docker service create --name web-app --replicas 2 -p 80:80 nginx
```

```
rihem@rihem-virtual-machine:~$ docker service create --name web-app --replicas 2 -p 80:80 nginx
4qq88rn81bdxjpfwa7uyvacer
overall progress: 0 out of 2 tasks
overall progress: 0 out of 2 tasks
overall progress: 0 out of 2 tasks
overall progress: 0 out of 2 tasks
overall progress: 0 out of 2 tasks
overall progress: 0 out of 2 tasks
overall progress: 1 out of 2 tasks
1/2: preparing [=====>]
2/2: running [=====>]
```

Explication:

Dans cet exemple :

- **--name web-app** : Nom du service.
- **--replicas 3** : Déployez 3 réplicas (conteneurs) de l'application Nginx.
- **-p 80:80** : Expose le port 80 de l'application sur le port 80 de l'hôte.

2- Vérifier le déploiement :

```
docker service ls
```

```
rihem@rihem-virtual-machine:~$ docker service ls
ID            NAME      MODE     REPLICAS  IMAGE       PORTS
4qq88rn81bdx web-app   replicated 2/2        nginx:latest *:80->80/tcp
```

Explication:

La commande **docker service ls** liste les services en cours d'exécution dans le Swarm. Vous devriez voir votre service **web-app** en cours avec les réplicas.

3- Inspecter le service web-app:

```
docker service inspect <ID/nom>
```

```
rihem@rihem-virtual-machine:~$ docker service inspect web-app
[
  {
    "ID": "4qq88rn81bdxjpfwa7uyvacer",
    "Version": {
      "Index": 39
    },
    "CreatedAt": "2025-02-10T16:37:42.203145452Z",
    "UpdatedAt": "2025-02-10T16:37:42.206032974Z",
    "Spec": {
      "Name": "web-app",
      "Labels": {},
      "TaskTemplate": {
        "ContainerSpec": {
          "Image": "nginx:latest@sha256:91734281c0ebfc6f1aea979cffe5079cfe786228a71cc6f1f46a228cde6e34",
          "Init": false,
          "StopGracePeriod": 10000000000,
          "DNSConfig": {},
          "Isolation": "default"
        },
        "Resources": {
          "Limits": {},
          "Reservations": {}
        },
        "RestartPolicy": {
          "Condition": "any",
          "Delay": 5000000000,
          "MaxAttempts": 0
        }
      }
    }
  }
]
```

4- Afficher les logs du service créer:

```
docker service logs web-app
```

```
rihem@rihem-virtual-machine: $ docker service logs web-app
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
web-app.1.9b5pznum7ttp@rihem-virtual-machine | /docker-entrypoint.sh: Configuration complete; ready for start up
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: using the "epoll" event method
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: nginx/1.27.4
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: OS: Linux 6.8.0-52-generic
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: start worker processes
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: start worker process 29
web-app.1.9b5pznum7ttp@rihem-virtual-machine | 2025/02/10 16:39:53 [notice] 1#1: start worker process 30
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-app.2.q5i96kyl45re@rihem-virtual-machine | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
web-app.2.q5i96kyl45re@rihem-virtual-machine | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
web-app.2.q5i96kyl45re@rihem-virtual-machine | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
```

Scaling des applications sur avec `docker service` :

Le scaling d'une application dans Docker Swarm permet d'ajuster le nombre de réplicas d'un service en fonction de la charge ou de la demande. Docker Swarm gère automatiquement la distribution de ces réplicas entre les nœuds disponibles, permettant ainsi d'augmenter ou de réduire la capacité de l'application sans interruption de service.

Exemple à pratiquer:

1- Augmenter la capacité de l'application et faire tourner 3 réplicas du service :

```
docker service update --replicas 3 web-app
```

```
rihem@rihem-virtual-machine: $ docker service update --replicas 3 web-app
web-app
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service web-app converged
```

2- Vérifier si le scaling a bien eu lieu:

```
docker service ps web-app
```

```
rihem@rihem-virtual-machine: $ docker service ps web-app
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
9b5pznum7ttp	web-app.1	nginx:latest	rihem-virtual-machine	Running	Running 28 minutes ago		
q5i96kyl45re	web-app.2	nginx:latest	rihem-virtual-machine	Running	Running 28 minutes ago		
416978bvlbqj	web-app.3	nginx:latest	rihem-virtual-machine	Running	Running 3 minutes ago		

3- Réduire la capacité du service:

```
docker service update --replicas 2 web-app
```

```
rihem@rihem-virtual-machine: $ docker service update --replicas 2 web-app
web-app
overall progress: 2 out of 2 tasks
1/2: running [=====>]
2/2: running [=====>]
verify: Service web-app converged
rihem@rihem-virtual-machine: $
```

Déploiement et Scaling Des applications sur Docker Swarm: docker stack



Introduction:

Dans cette section, nous allons explorer **Docker Stack**, un outil puissant permettant de déployer et gérer des applications multi-conteneurs sur un cluster **Docker Swarm**. Contrairement à docker service, qui gère des services individuellement, docker stack facilite le déploiement **d'applications complètes** définies dans un fichier *docker-compose.yml*.



Qu'est-ce qu'un stack?

Un **stack** dans Docker Swarm est un **ensemble de services** (conteneurs) définis et gérés ensemble comme une application unique. Il permet de déployer des **applications multi-conteneurs** en utilisant un fichier *docker-compose.yml*, qui définit les services, réseaux et volumes nécessaires.



`docker service` vs. `docker stack`:

Critère	docker service create	docker stack deploy
Utilisation	Crée un seul service dans Swarm.	Déploie une pile multi-services à partir d'un fichier Compose.
Portée	Un service à la fois.	Plusieurs services définis dans un fichier Compose.
Configuration par fichier	Pas de fichier nécessaire, configuration via la commande.	Utilise un fichier <i>docker-compose.yml</i> .
Swarm requis	Oui	Oui
Flexibilité	Moins flexible.	Plus flexible (fichier Compose).
Gestion des mises à jour	Mise à jour manuelle via <code>docker service update</code> .	Mise à jour via <code>docker stack deploy</code> .



La commande `docker stack`

Voici une liste des principales commandes ***docker stack*** utilisées pour gérer un stack dans Docker Swarm :

Commande	Description
<code>docker stack deploy -c docker-compose.yml <nom_du_stack></code>	Déploie une application à partir d'un fichier docker-compose.yml.
<code>docker stack ls</code>	Liste les stacks actifs sur le cluster Swarm.
<code>docker stack services <nom_du_stack></code>	Affiche les services d'un stack déployé.
<code>docker stack ps <nom_du_stack></code>	Liste les conteneurs en cours d'exécution pour un stack donné.
<code>docker stack rm <nom_du_stack></code>	Supprime un stack et tous ses services.

 **Remarque :** Les commandes `docker service` peuvent être utilisées en complément pour affiner la gestion des services au sein d'un stack.



Déploiement d'une application avec `docker stack` :

Voici un exemple simple d'utilisation de `docker stack` pour déployer une application composée d'un service web basé sur Nginx et un service de base de données MySQL.



Exemple à pratiquer:

1- Préparer le fichier `docker-compose.yml`:

```
1 version: "3.8"
2
3 services:
4   web:
5     image: nginx:latest
6     deploy:
7       replicas: 3
8       resources:
9         limits:
10          cpus: "0.5"
11          memory: "512M"
12     restart_policy:
13       condition: on-failure
14   ports:
15     - "80:80"
16
17   db:
18     image: mysql:5.7
19     environment:
20       MYSQL_ROOT_PASSWORD: example
21     deploy:
22       replicas: 1
23       resources:
24         limits:
25          cpus: "0.5"
26          memory: "512M"
27     restart_policy:
28       condition: on-failure
29
```



```
version: "3.8"
```

2- Déployer le stack à partir du fichier docker-compose.yml créé en utilisant la commande:

services:

```
web:
  image: nginx:latest
```

```
rihem@rihem-virtual-machine:~$ docker stack deploy -c docker-compose.yml appstack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service appstack_web
Creating service appstack_db
```

limits:

cpus: "0.5"

 **Remarque :** Si votre fichier docker-compose.yml se trouve dans le répertoire courant, vous pouvez omettre l'option -c et Docker trouvera automatiquement le fichier.

restart_policy:

condition: on-failure

ports:

"80:80"

3- Vérifier le déploiement:

```
docker stack services appstack
image: mysql:5.7
```

```
rihem@rihem-virtual-machine:~$ docker stack services appstack
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
z2gnawcqc5u5      appstack_db          replicated           1/1                  mysql:5.7
woapzj051ryl      appstack_web          replicated           3/3                  nginx:latest         *:80->80/tcp
```

deploy:

replicas: 1

4- Lister les stacks existants:

limits:

```
docker stack ls
```

```
rihem@rihem-virtual-machine:~$ docker stack ls
NAME                SERVICES
appstack            2
```

5- Lister les conteneurs en cours d'exécution pour le stack appstack:

```
docker stack ps appstack
```

```
rihem@rihem-virtual-machine:~$ docker stack ps appstack
ID                NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE    ERROR
PORTS
m58wyaw28abe      appstack_db.1        mysql:5.7            rihem-virtual-machine Running          Running 2 minutes ago
rs6pkyszyki       appstack_web.1       nginx:latest         rihem-virtual-machine Running          Running 2 minutes ago
klsugmotti0f      appstack_web.2       nginx:latest         rihem-virtual-machine Running          Running 2 minutes ago
qtocgmzjxvy8      appstack_web.3       nginx:latest         rihem-virtual-machine Running          Running 2 minutes ago
```

6- Supprimer le stack et vérifier:

Exemple1: Déployer et gérer un service nginx dans un Swarm avec Docker service

Introduction:

Dans cet exercice, vous allez déployer une application simple en utilisant l'image **nginx**. Pour configurer l'environnement de travail nous allons explorer la plateforme **Play with Docker (PWD)**, un environnement en ligne qui vous permet d'expérimenter avec Docker dans un environnement sécurisé, sans avoir besoin d'installer quoi que ce soit localement. Cette plateforme offre un moyen simple et interactif d'apprendre et de tester les concepts Docker en simulant un environnement Docker Swarm multi-nœuds et en exécutant diverses applications conteneurisées.

Commençons !

1- Accédez à la plateforme **Play with Docker** et créez un compte en cliquant sur [ce lien](https://labs.play-with-docker.com/) (<https://labs.play-with-docker.com/>).



2- Une fois cela fait, vous verrez cette interface. Cliquez sur 'Add new instance' pour lancer vos instances (dans cet exercice on a besoin de trois instances):

03:59:47

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

GIVE FEEDBACK

Add instances to your playground.

Sessions and all their instances are deleted after 03:59:47 hours.

+ ADD NEW INSTANCE

192.168.0.23
node1

192.168.0.22
node2

192.168.0.21
node3

GIVE FEEDBACK

SSH

ssh ip172-18-0-69-cuvkbviim2rg0090q8a0@direct.labs.pla:

DELETE EDITOR

```
[node1] (local) root@192.168.0.23 ~
$
```

3- Initialiser un Swarm (Manager):

```
docker swarm init --advertise-addr <IP_manager>
```

```
[node1] (local) root@192.168.0.23 ~
$ docker swarm init --advertise-addr 192.168.0.23
Swarm initialized: current node (9d8etpila57ydh24r42ndz7mt) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0o0bxb02qd55vx6ey6w1kc0d7jagwnneobz5ua3ojictl0puoi-0frfxupjqrf02hztptwptb4n6m 192.168.0.23:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

4- Ajouter deux nœuds Worker au Swarm:

```
docker swarm join --token <SWARM_JOIN_TOKEN> <IP_manager>:2377
```

- Pour copier tapez sur : *ctrl+insert*
- Pour coller tapez sur : *shift+insert*

```
[node2] (local) root@192.168.0.22 ~
$ docker swarm join --token SWMTKN-1-0o0bxb02qd55vx6ey6wlc0d7jagwnneobz5ua3ojict10puoi-0frfxupjqrfo2hztptwptb4n6m 192.168.0.23:2377
This node joined a swarm as a worker.
```

- Pour joindre le deuxième nœud worker (node2), il faut tout d'abord générer un token en lançant la commande suivante sur le nœud manager :

```
docker swarm join-token worker
```

```
[node1] (local) root@192.168.0.23 ~
$ docker swarm join-token worker
To add a worker to this swarm, run the following command:

docker swarm join --token SWMTKN-1-0o0bxb02qd55vx6ey6wlc0d7jagwnneobz5ua3ojict10puoi-0frfxupjqrfo2hztptwptb4n6m 192.168.0.23:2377
```

- Puis lancer la commande générée sur le nœud worker 2 (dans ce cas, node3) :

```
[node3] (local) root@192.168.0.21 ~
$ docker swarm join --token SWMTKN-1-0o0bxb02qd55vx6ey6wlc0d7jagwnneobz5ua3ojict10puoi-0frfxupjqrfo2hztptwptb4n6m 192.168.0.23:2377
This node joined a swarm as a worker.
```

5- Vérifier les nœuds du Swarm en tapant la commande:

```
docker node ls
```

```
[node1] (local) root@192.168.0.23 ~
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
9d8etpila57ydh24r42ndz7mt *	node1	Ready	Active	Leader	27.3.1
bt5eaq5husx3c0doqq76lge0v	node2	Ready	Active		27.3.1
vj8bjlpx5lwiecxof2m85m0x	node3	Ready	Active		27.3.1

6- Créer un service avec 4 réplicas, port mapping et limitation de mémoire:

```
docker service create --name web-service --replicas 4 -p 8080:80 --limit-memory 50m
nginx
```

```
[node1] (local) root@192.168.0.23 ~
$ docker service create --name web-service --replicas 4 -p 8080:80 --limit-memory 50m nginx
i77hg7jlja39q5g9gdc439ti
overall progress: 4 out of 4 tasks
1/4: running [=====>]
2/4: running [=====>]
3/4: running [=====>]
4/4: running [=====>]
verify: Service i77hg7jlja39q5g9gdc439ti converged
```

7- Vérification du service et des tâches:

```
docker service ps web-service
```

```
[node1] (local) root@192.168.0.23 ~
$ docker service ps web-service
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
g6sive8m9ob5	web-service.1	nginx:latest	node3	Running	Running 3 minutes ago		
5s2z2y3dgd01	web-service.2	nginx:latest	node3	Running	Running 3 minutes ago		
yew0o7g70v1b	web-service.3	nginx:latest	node1	Running	Running 3 minutes ago		
gji4zbuphs7s	web-service.4	nginx:latest	node2	Running	Running 3 minutes ago		

8- Accéder à votre application en utilisant la commande :

```
curl http://<Ip_manager>:8080
```

```
[node1] (local) root@192.168.0.23 ~
$ curl http://192.168.0.23:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
```

9- Mettre le noeud worker1 (node2) en mode drain et révéifier:

```
docker node update --availability drain <node_name>
```

```
[node1] (local) root@192.168.0.23 ~
$ docker node update --availability drain node2
node2
[node1] (local) root@192.168.0.23 ~
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
9d8etpila57ydh24r42ndz7mt *	node1	Ready	Active	Leader	27.3.1
bt5eaq5shusx3c0dogq761ge0v	node2	Ready	Drain		27.3.1
vj8bjlpkx5lwicqxf2m85m0x	node3	Ready	Active		27.3.1

```
[node1] (local) root@192.168.0.23 ~
$ docker service ps web-service
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
g6sive8m9ob5	web-service.1	nginx:latest	node3	Running	Running 32 minutes ago		
5s2z2y3dgd01	web-service.2	nginx:latest	node3	Running	Running 32 minutes ago		
yew0o7g70v1b	web-service.3	nginx:latest	node1	Running	Running 32 minutes ago		
p2hx4m6soa8v	web-service.4	nginx:latest	node1	Running	Running 24 minutes ago		
gji4zbuphs7s	web-service.4	nginx:latest	node2	Shutdown	Shutdown 24 minutes ago		

Le préfixe _ signale que cette tâche spécifique est une **ancienne instance du service qui n'est plus active**.

10- Mettre à jour le service web-service avec une nouvelle image et vérifier:

```
docker service update --image nginx:1.25.3 web-service
```

```
[node1] (local) root@192.168.0.13 ~
$ docker service update --image nginx:1.25.3 web-service
web-service
overall progress: 4 out of 4 tasks
1/4: running
2/4: running
3/4: running
4/4: running
verify: Service web-service converged
```

```
[node1] (local) root@192.168.0.13 ~
$ docker service ps web-service
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
86jcg2d5f6z	web-service.1	nginx:1.25.3	node1	Running	Running 30 seconds ago		
ti6zkg7j6p07	_ web-service.1	nginx:latest	node1	Shutdown	Shutdown 35 seconds ago		
na8cksutl8hw	web-service.2	nginx:1.25.3	node2	Running	Running 42 seconds ago		
jiou52p16ih5	_ web-service.2	nginx:latest	node2	Shutdown	Shutdown 49 seconds ago		
pv181a51490z	web-service.3	nginx:1.25.3	node3	Running	Running 53 seconds ago		
hv589e5kok4t	_ web-service.3	nginx:latest	node2	Shutdown	Shutdown about a minute ago		
k8qz9t51xdpz	web-service.4	nginx:1.25.3	node1	Running	Running 25 seconds ago		
ah62xbdm3ugb	_ web-service.4	nginx:latest	node3	Shutdown	Shutdown 26 seconds ago		

11- Promouvoir node2:

- Avant de le faire, il faut mettre à jour node2 en mode **Active** en tapant la commande suivante :

```
docker node update --availability active node2
```

```
[node1] (local) root@192.168.0.23 ~
$ docker node update --availability active node2
node2
[node1] (local) root@192.168.0.23 ~
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
9d8etpila57ydh24r42ndz7mt *	node1	Ready	Active	Leader	27.3.1
bt5eaq5husx3c0doqg761ge0v	node2	Ready	Active		27.3.1
vj8bj1px5lwicqxf2m85m0x	node3	Ready	Active		27.3.1

- Puis lancer cette commande:

```
docker node promote node2
```

```
[node1] (local) root@192.168.0.23 ~
$ docker node promote node2
Node node2 promoted to a manager in the swarm.
[node1] (local) root@192.168.0.23 ~
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
9d8etpila57ydh24r42ndz7mt *	node1	Ready	Active	Leader	27.3.1
bt5eaq5husx3c0doqg761ge0v	node2	Ready	Active	Reachable	27.3.1
vj8bj1px5lwicqxf2m85m0x	node3	Ready	Active		27.3.1

Explication:

Docker Swarm utilise l'**algorithme de consensus Raft** pour assurer la cohérence entre les nœuds managers. Lorsqu'il y a **plusieurs managers**, leur statut peut être :

1. **Leader** → Le manager principal, responsable de l'orchestration des tâches et de la prise de décisions pour tout le cluster.
2. **Reachable** → Un nœud manager qui fait partie du quorum Raft, mais qui n'est pas le leader. Il peut participer aux décisions, mais ne prend pas d'actions de leadership sauf en cas de besoin.

12- Rétrograder node1:

```
docker node demote node1
```

```
[node1] (local) root@192.168.0.23 ~  
$ docker node demote node1  
Manager node1 demoted in the swarm.
```

- Si on lance la commande `docker node ls` à partir de node1, on obtient l'affichage suivant :

```
[node1] (local) root@192.168.0.23 ~  
$ docker node ls  
Error response from daemon: This node is not a swarm manager. Worker nodes can't be used to view or modify cluster state. Please run this command on a manager node or promote the current node to a manager.
```

Explication:

Node1 n'est plus le nœud manager, et node2 est devenu le nouveau manager.

- Maintenant, si nous exécutons la commande suivante sur node2 :

```
docker node ls
```

```
[node2] (local) root@192.168.0.22 ~  
$ docker node ls  
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION  
9d8etpila57ydh24r42ndz7mt        node1       Ready     Active           Leader             27.3.1  
t5eaq5husx3c0doqq761ge0v *      node2       Ready     Active           Leader             27.3.1  
vj8bjlpx51wlecqxoizme3mux        node3       Ready     Active           Leader             27.3.1
```

=> On trouve que node2 est devenu le nœud manager Leader.

13- Nettoyer de l'environnement :

- Supprimer le service web-service:

```
docker service rm web-service
```

```
[node2] (local) root@192.168.0.22 ~
$ docker service rm web-service
web-service
[node2] (local) root@192.168.0.22 ~
$ docker service ps web-service
no such service: web-service
```

- Quitter le Swarm en exécutant la commande suivante sur chaque nœud worker et manager :

```
docker swarm leave --force
```

```
[node2] (local) root@192.168.0.22 ~
$ docker swarm leave --force
Node left the swarm.
```

```
[node1] (local) root@192.168.0.23 ~
$ docker swarm leave --force
Node left the swarm.
```

```
[node3] (local) root@192.168.0.21 ~
$ docker swarm leave --force
Node left the swarm.
```

Félicitations ! Vous avez maîtrisé les bases de Docker Swarm et la gestion des services. 🚀

Exemple2: Déployer une application WordPress avec Docker Stack

Introduction :

Dans le chapitre précédent, nous avons déployé une application WordPress en utilisant **Docker Compose** avec les services principaux suivants :

- **Une base de données MySQL.**
- **Le système de gestion de contenu WordPress** connecté à cette base.

Dans cet exercice, nous allons reprendre ce même déploiement, mais cette fois-ci en utilisant **Docker Stack** dans un environnement Docker Swarm. Nous allons configurer les mêmes services, volumes, réseaux et ports pour déployer l'application sur Docker Swarm.

Commençons !

1- Initialiser Docker Swarm: Avant de pouvoir déployer un stack, Docker Swarm doit être initialisé.

2- Créer un répertoire pour l'application et y accéder:

```
mkdir wordapp
```

```
rihem@rihem-virtual-machine:~$ mkdir wordapp
```

```
rihem@rihem-virtual-machine:~$ cd wordapp
rihem@rihem-virtual-machine:~/wordapp$
```

3- Créer le fichier docker-compose.yml pour Docker Stack:


```
1 version: "3.8"
2
3 services:
4   wordpress:
5     image: wordpress:latest
6     environment:
7       WORDPRESS_DB_HOST: db:3306
8       WORDPRESS_DB_NAME: wordpress
9       WORDPRESS_DB_USER: root
10      WORDPRESS_DB_PASSWORD: example
11     ports:
12       - "8082:80"
13     networks:
14       - my_network
15     deploy:
16       replicas: 1
17       restart_policy:
18         condition: on-failure
19
20   db:
21     image: mysql:5.7
22     environment:
23       MYSQL_ROOT_PASSWORD: example
24       MYSQL_DATABASE: wordpress
25     volumes:
26       - db_data:/var/lib/mysql
27     networks:
28       - my_network
29     deploy:
30       replicas: 1
31       restart_policy:
32         condition: on-failure
33
34 volumes:
35   db_data:
36
37 networks:
38   my_network:
39     driver: overlay
40
```

```

version: "3.8"

services:
  wordpress:
    image: wordpress:latest
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_USER: root
      WORDPRESS_DB_PASSWORD: example
    ports:
      - "8082:80"
    networks:
      - my_network
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: example
      MYSQL_DATABASE: wordpress
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - my_network
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure

volumes:
  db_data:

networks:
  my_network:
    driver: overlay

```

4- Déployer l'application avec Docker Stack:

```
docker stack deploy -c docker-compose.yml wordpress_stack
```

```

rihem@rihem-virtual-machine:~/wordapp$ docker stack deploy -c docker-compose.yml wordpress_stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wordpress_stack_my_network
Creating service wordpress_stack_wordpress
Creating service wordpress_stack_db

```

5- Vérifier le déploiement:

```
docker stack services wordpress_stack
```

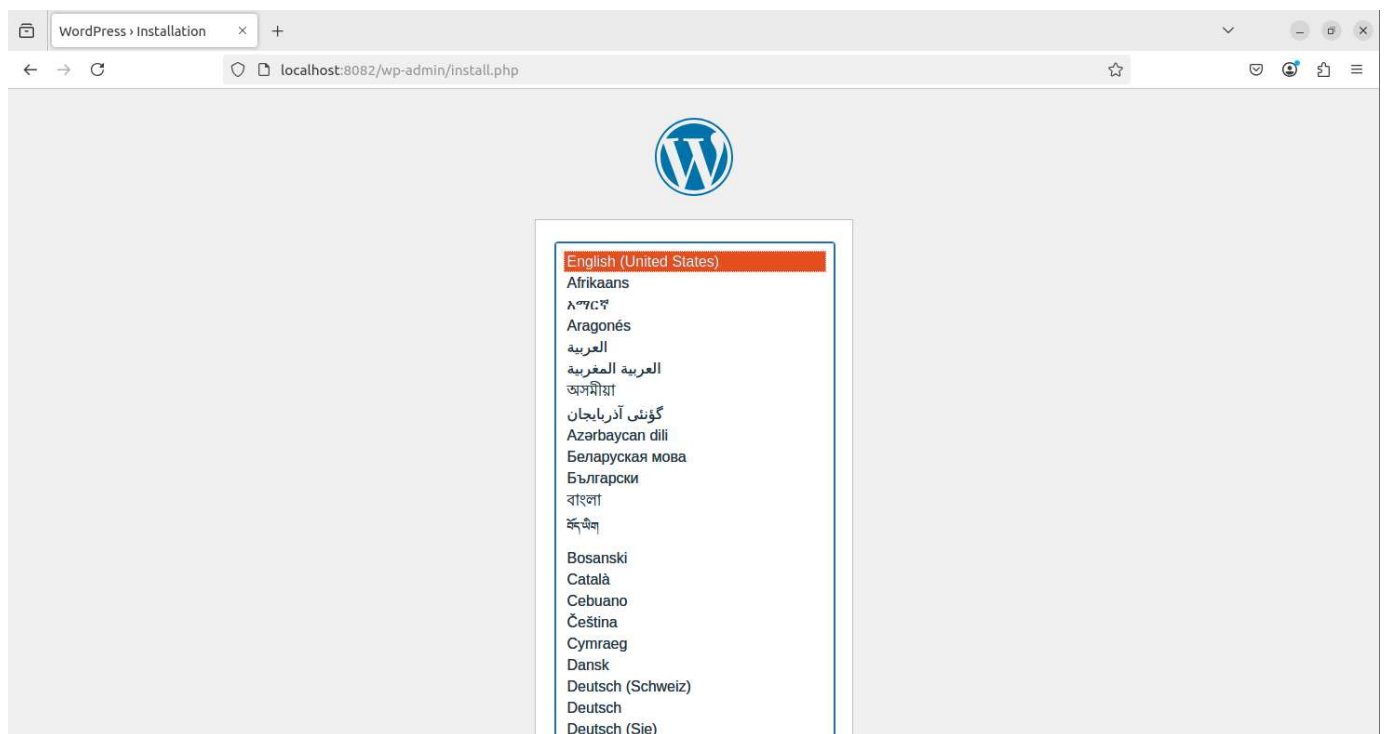
```
rihem@rihem-virtual-machine:~/wordapp$ docker stack services wordpress_stack
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
l9sbp9a9ga95	wordpress_stack_db	replicated	1/1	mysql:5.7	
dhw26ebt2rmy	wordpress_stack_wordpress	replicated	0/1	wordpress:latest	*:8082->80/tcp

6- Accéder à l'application WordPress:

Ouvrir votre navigateur et visiter l'adresse suivante pour configurer ton site WordPress :

<http://127.0.0.1:8082> (<http://127.0.0.1:8082/>)



7- Mettre à jour la version de l'image wordPress:

- Lister les services déployés:

```
rihem@rihem-virtual-machine:~/wordapp$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
l9sbp9a9ga95	wordpress_stack_db	replicated	1/1	mysql:5.7	
dhw26ebt2rmy	wordpress_stack_wordpress	replicated	1/1	wordpress:latest	*:8082->80/tcp

- Ouvrir votre fichier docker-compose.yml et modifier l'image utilisée par le service WordPress:

```
1 version: "3.8"
2
3 services:
4   wordpress:
5     image: wordpress:6.0
6     environment:
7       WORDPRESS_DB_HOST: db:3306
8       WORDPRESS_DB_NAME: wordpress
9       WORDPRESS_DB_USER: root
10      WORDPRESS_DB_PASSWORD: example
11     ports:
12       - "8082:80"
13     networks:
14       - my_network
15     deploy:
16       replicas: 1
17       restart_policy:
18         condition: on-failure
19
20   db:
21     image: mysql:5.7
22     environment:
23       MYSQL_ROOT_PASSWORD: example
24       MYSQL_DATABASE: wordpress
25     volumes:
26       - db_data:/var/lib/mysql
27     networks:
28       - my_network
29     deploy:
30       replicas: 1
31       restart_policy:
32         condition: on-failure
33
34 volumes:
35   db_data:
```